

Load images(#3) loads the image that you would use for the bunny.

Keep looping through(#4) keeps looping over the following indented code.

Clear the screen(#5) clears the screen. Fill the screen with black before you draw anything.

Draw the screen element(#6) adds the bunny image that you loaded to the screen at x= 100 and y= 100.

Update the screen(#7) updates the screen.

Loop through the event (#8) checks for any new events; if there is one, and if it is quit command, exit the program.

If you run the code now (via Run\ Run Module in Idle menu), you would see a screen as shown in Fig. 1. Woot, the bunny, is on the scene and ready for action.

Add scenery. Add a background to the game scene, which can be done with some more screen.blit() calls.

At the end of load image section #3 above, after loading the player image, add the following code:

```
grass = pygame.image.load ("resources/
images/grass.png")
castle = pygame.image.load ("resources/
images/castle.png")
```

This loads the images of grass and castle, and puts these into specific variables. But if you check the grass image, you would notice that it does not cover the entire screen area, which is 640x480.

Add the following code to game.py at the beginning of draw the screen section #6 (before the player is drawn on the screen):

```
for x in range(width/grass.get_
width()+1):
    for y in range(height/grass.
get_height()+1):
        screen.blit(grass,
            (x*100,y*100))
screen.blit(castle, (0,30))
screen.blit(castle, (0,135))
screen.blit(castle, (0,240))
screen.blit(castle, (0,345 ))
```

As you can see, the 'for' statement loops through x first. Then,

within that loop, it loops through y and draws the grass at x and y values generated by 'for' loop. The next couple of lines draw castles on the screen. If you run the program now, you would get something like shown in Fig. 2.

Make the bunny move. Next, add some actual gameplay elements, like making the bunny respond to key presses. To do that, implement a method of keeping track of which keys are being pressed at any given moment. You can do this simply by making an array of key states that hold the state of each key you want to use for the game.



Fig. 1: Bunny on the screen

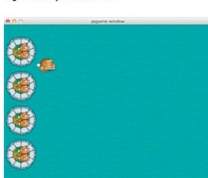


Fig. 2: Bunny with background colour

Add the following code to game.py at the end of initialise the game section #2 (after you set the screen height and width):

```
keys = [False, False, False, False]
playerpos=[100,100]
```

This code is self-explanatory. The keys array keeps track of the keys being pressed in the following order: WASD. Each item in the array corresponds to one key with particular action—first to W for up, second to A for left, S for down and

D for right movement.

Playerpos variable is where the program draws the player (bunny).

You now need to modify the existing code for drawing the player to use the new playerpos variable. Change the following lines in draw the screen section:

```
screen.blit(player, (100,100))
to
screen.blit(player, playerpos)
```

Update the keys array based on the keys being pressed. PyGame makes key press detection easy by adding event.key functions.

At the end of loop through event section #8, right after the block checking for event, type pygame.QUIT. Put the following code at the same indentation level as pygame.QUIT block:

```
if event.type == pygame.KEYDOWN:
    if event.key==K_w:
        keys[0]=True
    elif event.key==K_a:
        keys[1]=True
    elif event.key==K_s:
        keys[2]=True
    elif event.key==K_d:
        keys[3]=True
if event.type == pygame.KEYUP:
    if event.key==pygame.K_w:
        keys[0]=False
    elif event.key==pygame.K_a:
        keys[1]=False
    elif event.key==pygame.K_s:
        keys[2]=False
    elif event.key==pygame.K_d:
        keys[3]=False
```

Check to see if a key is being pressed down or released. If the key being pressed or released is one of the keys you are using, update the keys variable accordingly.

Finally, update playerpos variable in response to the key being pressed. This is quite simple. Add the following code to the end of game.py (with one indentation level, putting it at the same level as 'for' loop):

```
# 9 - Move player
if keys[0]:
    playerpos[1]=5
elif keys[2]:
```